STRIPE PROJECT

3. NoSQL Data System

D4. NoSQL Data Model

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D4 : NoSQL Data Model / v08 --- 2026/11/05.*

D4.1 Contexte

Le *business case* [R0] demande que le système *NoSQL* couvre les données semi-structurées et non structurées issues des sources suivantes (cf. [R0] 3. *Data Source* / *Semi-Structured and Non Structured Data*) :

- interactions avec les utilisateurs (clickstreams et données de session),
- features d'apprentissage automatique (détection de fraude),
- retours clients (avis, réponses d'enquête).
- journaux.

Compte tenu des exigences techniques du *business case* (*TR1*, en particulier), le choix d'un système de stockage *par documents* s'impose (cf. justification en annexe §3.A).

Définition des collections en Python

La création des collections, des indexes et des partitionnements *sharding* est exprimée en *Python*, exécutée par un objet `db`, dédié aux accès à la base `stripe`, et instancié par la classe `MongoClient` du module `pymongo` :

```python
from pymongo import MongoClient

client = MongoClient("mongodb://...")   # connexion au serveur
db = client["stripe"]                   # accès client à la base "stripe"
```

D4.2. Méthode d'analyse

La méthodologie de référence proposée *MongoDB* [A1] consiste à identifier d'abord les motifs d'accès générés par l'application, puis, dans un second temps seulement, à examiner la forme physique des documents (comme conséquence des motifs d'accès).

D4.3. Principes de conception

Le modèle *NoSQL* est gouverné par les principes suivant.

1. Modéliser selon les accès

En relationnel, on modélise le domaine puis on écrit les requêtes. En document, on part des requêtes : chaque collection est conçue pour qu'une lecture typique n'implique *aucune jointure*.

2. Schéma flexible mais contrôlé

L'absence de schéma imposé permet d'absorber des structures hétérogènes (un log d'erreur n'a pas les mêmes champs qu'un log d'accès). On applique néanmoins une validation *JSON Schema* sur les champs critiques pour éviter une dérive silencieuse.

3. Imbriquer par défaut, référencer exceptionnellement

Le levier de performance principal du modèle orienté *documents*, c'est d'éviter les jointures. Un document qui *embarque* (par opposition à "référencer") toutes les données que l'on attend d'une lecture typique permet de récupérer l'ensemble de ces données en un seul accès disque. L'imbrication est donc le choix de relation par défaut. Une entité interne d'un document n'est référencée que dans certains cas particuliers : entité partagée entre plusieurs documents, entité de taille non bornée, entité dont le cycle de vie est indépendant du document qui la mentionne.

D4.4. Analyse des motifs d'accès

Collection	Écriture	Lecture	Ratio écriture/lecture
`event_logs`	Continue, flux massif	Par fenêtre temporelle	Largement dominé par l'écriture
`user_sessions`	Unitaire, à chaque interaction	Session entière, en une fois	Fortement dominé par l'écriture
`ml_features`	Recalcul périodique par entité	Immédiate par le modèle (inférence temps réel)	Équilibré, lecture critique en latence
`customer_feedback`	Occasionnelle (après achat/interaction)	Par client, ou agrégée par marchand/produit	Dominé par la lecture agrégée
`merchant_config`	Quasi jamais	Très fréquente, par clé unique	Quasi exclusivement lu +

Cette analyse commande directement les décisions *embed* / *reference* et les index des sections suivantes (ex: un motif d'accès mal identifié ici produirait un schéma qu'aucun index ne pourrait ensuite corriger efficacement).

D4.5. Collections

D4.5.1. Remarque sur la validation des schémas

Les collections suivante sont soumises à une validation *JSON Schema* minimale, qui porte sur l'*enveloppe* du document (version, identifiants, horodatages, etc.) mais pas sur son contenu variable (`context` , `features` , `events` , `metadata`).

D4.5.2. `event_logs` : les journaux applicatifs

Volume très élevé, écriture massive, lecture ciblée par fenêtre temporelle.
Le sous-document `context` reste *libre* (forme variable selon le service émetteur).

```
from datetime import datetime, timezone, timedelta
now = datetime.now(timezone.utc) # ex : datetime(2026, 7, 23, 14, 32, 11, 482000)

# Document example :
# -- _id = automatically generated by the MongoDB's system
event_log_example = {
    "schema_version": 1,
    "timestamp": now,
    "level": "error",
    "service": "payment-gateway",
    "merchant_id": "a3f1...",
    "transaction_id": "b7c2...",
    "message": "Card authorization timeout",
    "context": { # -- structure libre (variable selon le service émetteur) :
        "http_status": 504,
        "latency_ms": 30012,
        "retry_count": 2,
        "upstream": "acquirer-eu-1"
    },
    "trace_id": "9f8e7d...",
    # -- TTL : purge automatique après 90 jours.
    "expires_at": now + timedelta(days=90)
}

# Schema validation :
event_logs_validator = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["schema_version", "timestamp", "level", "service", "merchant_id"],
        "properties": {
            "schema_version": {"bsonType": "int"},
            "timestamp":      {"bsonType": "date"},
            "level":          {"bsonType": "string"},
            "service":        {"bsonType": "string"},
            "merchant_id":    {"bsonType": "string"}
        }
    }
}

# Collection :
db.create_collection("event_logs", validator=event_logs_validator)
```

```
# Indexes :
db.event_logs.create_index([("timestamp", -1)])
db.event_logs.create_index([("merchant_id", 1), ("timestamp", -1)])
db.event_logs.create_index([("level", 1), ("timestamp", -1)])
# -- 'sparse' index ('transaction_id' field is mostly absent) :
db.event_logs.create_index([("transaction_id", 1)], sparse=True)
# -- TTL :
db.event_logs.create_index([("expires_at", 1)], expireAfterSeconds=0)

# Sharding :
# -- shardin key : { service: 1, timestamp: 1 }
db.command("shardCollection", "stripe.event_logs",
    key={"service": 1, "timestamp": 1})
```

Référencés vs. Embarqués

- Références : `merchant_id`, `transaction_id` ;
- Embarqués : `context` est une donnée embarquée (toujours lue avec son contexte parent, ici un log d'évènement).

Durée de vie

Purge automatique de l'entrée de journal 90 jours après sa création (principe de minimisation des données RGPD).

D4.5.3. `user_sessions` : clickstream et interactions

Une session d'utilisateur est un document.
Les événements sont *imbriqués*, car on ne consulte jamais un évènement de clic isolément.

```
from datetime import datetime

# Document example :
# -- _id = automatically generated by the MongoDB's system
user_session_example = {
    "_id": "sess_8f2a...",  # explicit customized id
    "schema_version": 2,
    "customer_id": "c91d...",
    "merchant_id": "a3f1...",
    "started_at": datetime(2026, 7, 23, 10, 2, 0), # TTL
    "ended_at": datetime(2026, 7, 23, 10, 19, 44), # TTL
    "duration_sec": 1064,
    "event_count": 4,
    "device": {
        "type": "mobile", "os": "iOS 18.2",
        "browser": "Safari", "screen": "390x844"
    },
    "geo": {"ip_country": "FR", "region": "Bretagne", "city": "Rennes"},
    "events": [
        {"ts": datetime(2026, 7, 23, 10, 2, 4), "type": "page_view",
         "metadata": {"path": "/checkout"}},
        {"ts": datetime(2026, 7, 23, 10, 6, 30), "type": "purchase",
         "metadata": {"transaction_id": "b7c2..."}}
    ],
    "outcome": "converted"
}

# Schema validation :
user_sessions_validator = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["schema_version", "customer_id", "merchant_id", "started_at"],
        "properties": {
            "schema_version": {"bsonType": "int"},
            "customer_id":    {"bsonType": "string"},
            "merchant_id":    {"bsonType": "string"},
            "started_at":     {"bsonType": "date"}
        }
    }
}
```

```
# Collection :
db.create_collection("user_sessions", validator=user_sessions_validator)

# Indexes :
db.user_sessions.create_index([("customer_id", 1), ("started_at", -1)])
db.user_sessions.create_index([("outcome", 1)])
# -- multikey :
db.user_sessions.create_index([("events.metadata.transaction_id", 1)])
# -- TTL (90 days) :
db.user_sessions.create_index([("started_at", 1)], expireAfterSeconds=90 * 24 * 3600)

# Sharding :
# -- shardin key : { merchant_id: 1, started_at: 1 }
db.command("shardCollection", "stripe.user_sessions",
           key={"merchant_id": 1, "started_at": 1})
```

Patterns *MongoDB*

- **Bucket Pattern** :
  - Un document par session, événements embarqués dans un tableau borné par la durée réelle de la session.
- **Polymorphic Pattern** :
  - `metadata` change de forme selon `type` (un évènement de porte un `target`, une erreur de formulaire porte un `field`, etc.).
- **Computed Pattern** :
  `event_count` et `duration_sec` sont pré-calculés à l'écriture pour une relecture directe.
- **Outlier Pattern** :
  - Un document *MongoDB* plafonne à 16 Mo. Une session très longue pourrait dépasser cette taille. On borne donc `events` (par exemple : 1000 entrées) et on bascule le surplus dans un document référencé : `user_sessions_overflow`. C'est le motif dit "*pattern outlier*".

Références vs. Embarqués

- Références : `customer_id`, `merchant_id`.
- Embarqués : `events`, `device`, `geo`.

Durée de vie

- Purge de la, session 90 jours après son démarrage (une fois que la session est agrégée et transportée vers la base OLAP, la donnée brute n'a pas vocation à être conservée indéfiniment).

D4.5.4. `ml_features` : le magasin des features

Alimente la détection de fraude en temps réel. Lecture à très faible latence sur la clé primaire.
une seule lecture doit suffire à disposer de toutes les features nécessaires à la décision, sans agrégation ni jointure à la volée.

Point de conception : un document continuellement actualisé par entité. C'est le bon grain pour un magasin de features servant l'inférence en temps réel : au moment où une nouvelle transaction arrive, il faut récupérer le profil *courant* du client (pas l'historique des évaluations passées). Le magasin de features *NoSQL* est donc un cache d'état courant.

```
from datetime import datetime

# Document example :
ml_features_example = {
    "_id": "cust:c91d...",  # explicit customized id
    "schema_version": 3,
    "entity_type": "customer",
    "entity_id": "c91d...",
    "merchant_id": "a3f1...",
    "computed_at": datetime(2026, 7, 23, 14, 0, 0),
    "features": {
        "txn_count_24h": 7,
        "txn_amount_sum_24h": 842.50,
        "distinct_countries_7d": 3,
        "distinct_cards_30d": 2,
        "avg_ticket_90d": 61.30,
        "failed_ratio_7d": 0.21,
        "hours_since_last_txn": 0.4,
        "device_switch_count_24h": 2
    },
```

```
    "model_scores": {
        "fraud_v3": 0.87,
        "churn_v1": 0.12
    }
}

# Schema validation :
ml_features_validator = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["schema_version", "entity_type", "entity_id", "merchant_id", "computed_at"],
        "properties": {
            "schema_version": {"bsonType": "int"},
            "entity_type":    {"bsonType": "string"},
            "entity_id":      {"bsonType": "string"},
            "merchant_id":    {"bsonType": "string"},
            "computed_at":    {"bsonType": "date"}
        }
    }
}

# Collection :
db.create_collection("ml_features", validator=ml_features_validator)

# Indexes :
db.ml_features.create_index([("merchant_id", 1), ("computed_at", -1)])
db.ml_features.create_index([("model_scores.fraud_v3", -1)])

# Sharding :
# -- shardin key : { _id: "hashed" }
db.command("shardCollection", "stripe.ml_features", key={"_id": "hashed"})
```

Patterns *MongoDB*

- **Schema Versioning Pattern** : `schema_version` : l'ensemble de features évolue à chaque itération de modèle sans bloquer les documents déjà écrits avec une version antérieure.
- **Computed Pattern** : les moyennes/compteurs glissants sont pré-calculés par un job périodique (et non pendant l'inférence, où la latence est critique).

Référencés vs. Embarqués

- Références : `entity_id` , `merchant_id` .
- Embarqués : `features` ~~+ `model_scores` (toujours lus ensemble)~~.

D4.5.5. `customer_feedback` : les réponses des avis et des enquêtes

Volume modeste, structure hétérogène, recherche plein texte.

```
from datetime import datetime

# Document example :
# -- _id = automatically generated by the MongoDB's system
customer_feedback_example = {
    "schema_version": 1,
    "merchant_id": "a3f1...",
    "customer_id": "c91d...",
    "type": "survey_response",
    "submitted_at": datetime(2026, 7, 20, 9, 14, 0),
    "channel": "email",
    "content": {
        "rating": 4,
        "text": "Le paiement a fonctionné, mais la confirmation a mis du temps.",
        "language": "fr"
    },
    "context": {
        "order_id": "txn_5a02...",
        "responses": [
            {"question_id": "q1", "answer": "4"},
            {"question_id": "q2", "answer": "Confirmation lente"}
        ]
```

```
    },
    "nlp": {
        "sentiment": "mixed",
        "sentiment_score": 0.12,
        "topics": ["latency", "confirmation"],
        "model_version": "sent-v2"
    }
}

# Schema validation :
customer_feedback_validator = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["schema_version", "merchant_id", "customer_id", "submitted_at"],
        "properties": {
            "schema_version": {"bsonType": "int"},
            "merchant_id":    {"bsonType": "string"},
            "customer_id":    {"bsonType": "string"},
            "submitted_at":   {"bsonType": "date"}
        }
    }
}

# Collection :
db.create_collection("customer_feedback", validator=customer_feedback_validator)

# Indexes :
db.customer_feedback.create_index([("merchant_id", 1), ("submitted_at", -1)])
db.customer_feedback.create_index([("customer_id", 1)])
db.customer_feedback.create_index([("content.text", "text")])
db.customer_feedback.create_index([("nlp.sentiment", 1)])

# Sharding :
# -- shardin key : { merchant_id: 1 }
db.command("shardCollection", "stripe.customer_feedback", key={"merchant_id": 1})
```

Patterns *MongoDB*

- **Polymorphic Pattern** : un avis noté et une réponse d'enquête NPS partagent la même collection sans partager la même forme de `content`.
- **Attribute Pattern** : les métadonnées variables selon le canal sont regroupées dans `context` plutôt que multipliées en champs racine optionnels.

Référencés vs. Embarqués

- Références : `customer_id`, `merchant_id`, `order_id`.
- Embarqués : `content`, `context`, `nlp`.

Stratégie d'indexation

- Indexes :
  - `{ merchant_id: 1, submitted_at: -1 }` pour le tableau de bord marchand,
  - `{ customer_id: 1 }` pour historique client, droit d'accès RGPD,
  - index texte sur `content.text` (recherche/analyse de sentiment),
  - `{ "nlp.sentiment": 1 }`

Stratégie de sharding

- Clé de *sharding* : `merchant_id` car lecture dominante du document par marchand.

D4.5.6. `merchant_config` : configuration dénormalisée

Cache de référence pour éviter d'interroger les tables transactionnelles à chaque événement applicatif. Petite collection, très lue, quasiment jamais écrite.

```
from datetime import datetime

# Document example :
merchant_config_example = {
```

```
    "_id": "a3f1...",  # explicit customized id
    "schema_version": 1,
    "legal_name": "Librairie du Désert",
    "country_code": "FR",
    "status": "active",
    "risk_profile": {
        "tier": "standard",
        "manual_review_threshold": 0.75,
        "auto_block_threshold": 0.95
    },
    "enabled_features": ["3ds", "subscriptions", "instant_payout"],
    "synced_at": datetime(2026, 7, 23, 6, 0, 0)
}

# Schema validation :
merchant_config_validator = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["schema_version", "legal_name", "status"],
        "properties": {
            "schema_version": {"bsonType": "int"},
            "legal_name":     {"bsonType": "string"},
            "status":         {"bsonType": "string"}
        }
    }
}

# Collection :
db.create_collection("merchant_config", validator=merchant_config_validator)

# Indexes :
# -- no indexes

# Sharding :
# -- no sharding
```

Patterns *MongoDB*

- **Extended Reference Pattern** : `_id` est la clé de métier transactionnelle (`merchant_id`), mais le document duplique quelques attributs fréquemment lus (`legal_name` , `status`) pour éviter une lecture croisée vers les table transactionnelle à chaque événement. La redondance est acceptable ici précisément parce que ces attributs changent rarement et que `synced_at` documente leur actualité.

Référencés vs. Embarqués

- Seule collection où des attributs normalement référencés (le nom du marchand) sont volontairement dupliqués pour des raisons de performance de lecture.

Stratégie d'indexation

- Indexes : pas d'index de secondaire car on accède directement par `_id` .

Stratégie de sharding

- Collection non "shardée" car le volume de données est trop faible (cf. §D4.7).

D4.6. Stratégies de relations

Pour répondre aux exigences d'imbrication, de référencement et d'indexation du *business case* [R0], on applique les règles de relations suivantes.

1. Imbrication (embedding)

Retenue quand les données sont lues ensemble, bornées en taille et appartiennent au parent.

Exemples : `events` d'une session, `responses` d'une enquête, ou le sous-document `device` .

> **Avantage** : pas de jointure, une seule lecture disque.
> C'est le principal levier de performance en *NoSQL* (cf. §D4.3/3).

2. Référencement

Retenu quand l'entité est partagée, volumineuse ou que son cycle de vie est indépendant.

Exemples : `transaction_id`, `merchant_id` et `customer_id` référencent les entités transactionnelles.

> **Point important** : ces références sont logiques et ne sont pas des contraintes.
> Il n'y a pas d'intégrité référentielle garantie par le moteur *NoSQL*.
> C'est le prix de la flexibilité, et cela impose que la base OLTP reste la source de vérité pour ces entités.

3. Pattern « extended reference »

Compromis employé dans `event_logs` et `user_sessions` : en plus de la référence (ex: `merchant_id`), on embarque une copie des quelques attributs fréquemment affichés (le nom du marchand, par exemple). On évite ainsi une lecture supplémentaire, au prix d'une redondance acceptable sur des données qui changent rarement (mais qui sont souvent lues).

4. Indexation

Types d'indexation disponibles et leur usage dans le modèle :

Type	Exemple	Usage dans le modèle
Simple / composé	`{merchant_id: 1, timestamp: -1}`	filtre + tri fréquents
Multikey	`events.metadata.transaction_id`	sur un tableau imbriqué
Texte	`context.text`	recherche plein texte
TTL	`event_logs.expires_at`	purge automatique
Sparse	`{transaction_id: 1}`	le champ n'existe que sur certains logs

Même arbitrage qu'en transactionnel : chaque index accélère la lecture et pénalise l'écriture.
Par exemple, on minimise volontairement les indexes sur le document `event_logs` car celui-ci doit supporter un flux d'écriture intense.

D4.7. Stratégie de *sharding*

Principe directeur : le *cloisonnement par marchand* sert de fil conducteur tant que le motif d'accès dominant n'impose pas une autre clé (principe déjà appliqué dans la base transactionnelle ; cf. [D2-C]).

> Rappel : Le *cloisonnement entre marchands* est le principe selon lequel *les données d'un marchand ne se mélangent jamais avec celles d'un autre* , et ce à tous les niveaux du système. Par exemple, un client qui achète chez trois marchands différents n'est pas, dans le modèle, "une seule personne pour trois achats mais trois enregistrements `Customer` distincts, un par marchand, sans lien direct entre eux dans les données transactionnelles.

Collection	Clé de sharding	Justification
`event_logs`	`{ service: 1, timestamp: 1 }`	Écriture répartie par service émetteur, lecture par fenêtre temporelle
`user_sessions`	`{ merchant_id: 1, started_at: 1 }`	Cloisonnement marchand préservé
`ml_features`	`{ _id: "hashed" }`	Répartition uniforme, accès par clé unique sans pattern marchand dominant
`customer_feedback`	`merchant_id`	Lecture dominante par marchand
`merchant_config`	Non "shardée"	Volume trop faible

D4.8. Intégration OLTP / OLAP

Le modèle *NoSQL* s'intègre aux deux autres systèmes de données de la manière suivante (voir aussi [D1]).

Intégration *OLTP > NoSQL*

Dans le système *OLTP*, les captures *CDC* sur `transaction`, `merchant` et `customer` alimentent un flux d'évènements *Kafka*, consommés par un agent qui actualise `merchant_config` et déclenche le recalcul des `ml_features`. Les identifiants *OLTP* (`transaction_id`, `merchant_id`) servent de clés de rapprochement.

Intégration *NoSQL > OLAP*

Les flux d'évènements *Change Streams* du système *NoSQL* assurent le transport continu des changements sur les collections `user_sessions` et `ml_features` vers les processus *Airflow*. Le consommateur *Airflow* accumule les changements reçus en continu et les transforme à échéance planifiée. Les sessions et les scores de fraude sont donc agrégés, et chargés dans `FactFraudEvent` et les

dimensions comportementales, par *lots nocturnes*. Les données brutes reçues restent dans le système *NoSQL* et seul les agrégats partent vers l'entrepôt analytique.

NoSQL comme couche de rapprochement transverse

C'est dans le système *NoSQL* que l'on peut rapprocher une même personne physique agissant chez plusieurs marchands (via l'empreinte de sa carte), sans compromettre le cloisonnement transactionnel entre marchands.

D4.9. Cohérence, sécurité et conformité

Modèle de cohérence

La vérification de cohérence à terme (*eventual consistency*) est acceptable pour les journaux et les sessions (un délai de quelques secondes sera sans conséquence). Inversement, dans le processus de décision de fraude, la contrainte de cohérence est très forte sur les données d'entrainement. Le système impose donc la garantie de cohérence maximale (`readConcern: "majority"`) aux opérations de lecture des données `ml_features` .

> Remarque : `readConcern` est un réglage *MongoDB* qui définit la garantie de cohérence des données lues. `"majority"` est le niveau le plus strict : il garantit que la donnée lue a été confirmée par la majorité des membres du *replica set* (l'ensemble des répliques d'un même jeu de données), et qu'elle ne sera donc pas annulée par un rollback ultérieur.

> Cette garantie a un coût : la lecture est plus lente qu'une lecture sur un seul nœud, puisqu'il faut attendre la confirmation de plusieurs répliques plutôt que d'en interroger une seule immédiatement disponible.

Chiffrement

Chiffrement au repos (au niveau du stockage) et en transit (*TLS*). Les champs sensibles (`text` des retours clients, données d'identification) relèvent du chiffrement au niveau du champ (*Client-Side Field Level Encryption*).

RGPD

Le droit à l'effacement impose de pouvoir supprimer toutes les traces d'un client. Les indexes sur `customer_id` présents dans chaque collection rendent cette opération réalisable. Les indexes *TTL* assurent par ailleurs une limite de conservation aux journaux.

PCI-DSS

Aucune donnée de carte bancaire complète n'est stockée dans le système *NoSQL*, conformément au principe déjà appliqué en *OLTP*.

Contrôle d'accès

Rôles définis par collection (lecture seule sur `event_logs` pour les analystes, écriture réservée aux services), et journalisation des accès.

D4.10. Couverture des exigences

#	Exigence	Couverture
BR3	Schéma flexible, types de données variés, requêtage de données imbriquées	Cinq collections, patterns Polymorphic/Attribute (§D4.5)
TR1	Schéma NoSQL flexible et requêtable	Index ciblés (§D4.6), validation minimale non bloquante (§D4.5.1)
TR3	Bases distribuées, sharding	Clé de sharding justifiée par collection (§D4.7)

Références

Ressources

- [R0] Stripe Business Case

Livrables

- [D1] Data Architecture Diagram

Annexes

- [D2-B] OLTP Physical Model
- [D2-C] OLTP Supporting Notes
- [D4-A] NoSQL Supporting Notes

Applicables

- [A1] MongoDB : Designing Your Schema
- [A2] MongoDB : Schema Design Patterns